

Class 02: JavaScript (JS), TypeScript (TS)

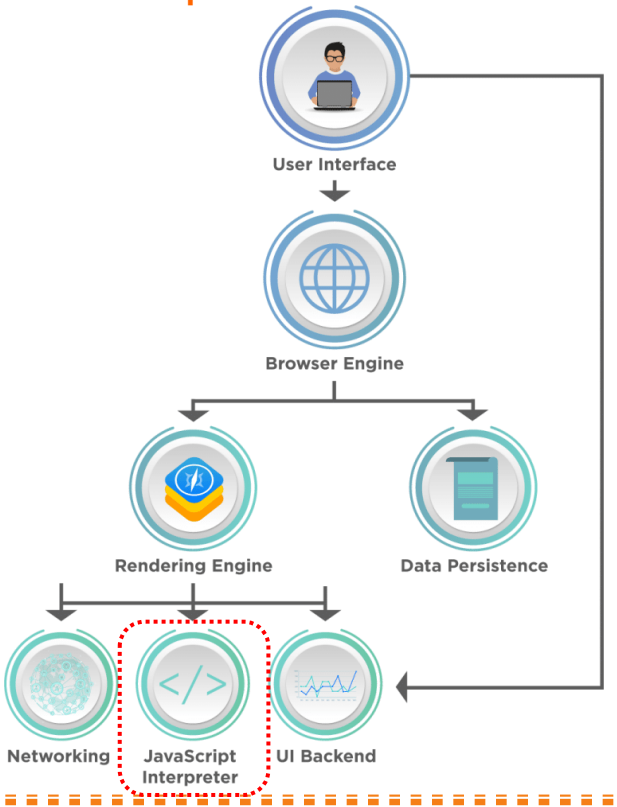
2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

JavaScript (JS)

- JavaScript เป็นภาษาภายใต้ ECMA standard จึงมีชื่อทางการว่า ECMAScript (ES)
  - 1997-1999 Original JavaScript ES1, ES2, ES3
  - 2009 ES5
  - 2015 ES6
    - ปัจจุบันเป็น ES6 โดยเรียกเวอร์ชั่นตามปีที่มีการอับเดตเวอร์ชั่น เช่น ECMAScript 2016, ECMAScript 2017, ECMAScript 2019, ECMAScript 2020 เป็นต้น

- JavaScript เป็น Scripting Language (interpreted language) ไม่ต้องการการ compile ก่อนการ run โดย Web Browser มี JavaScript interpreter จึงรองรับการประมวลผล JavaScript





Basic Syntax

- JavaScript เป็นภาษาที่ Case Sensitive
  - นิยมใช้ Camel Case ที่ขึ้นต้นด้วยตัวเล็กในการประกาศตัวแปร เช่น `firstName`, `lastName`
- JavaScript statements ลงท้ายด้วย semi-colon ;

```
var x, y;
x = 5 + 6;
y = x * 10;
```

  - การลงท้ายด้วย ไม่ได้บังคับ แต่แนะนำให้ใช้
- การเขียน comment (คำอธิบายโปรแกรม)

```
//comment

/*
Comment line 1
Comment line 2
*/
```

ข้อความ comment

JavaScript ES6 : Variable



- ES6 มาพร้อมฟีเจอร์ต่าง ๆ เช่น Constants, Block-Scoped Variables และ Functions, Arrow Functions, Default Function Parameters ฯลฯ
- ประกาศตัวแปร

```
variable_keyword  variable_name;
```

เช่น let หรือ var

- Scope of Variables
  - `var` ตัวแปรระดับ function (function-scoped) หรือ global scope

```
function test() {
  for(var i=0; i<3; i++)
  {
    console.log(i)
  }
  console.log(i)
}

test()
```

```
Console
0
1
2
3
```

  - `let` ตัวแปรระดับ Block (block-scoped)

```
function test() {
  for(let i=0; i<3; i++)
  {
    console.log(i)
  }
  console.log(i)
}

test()
```

```
Console
0
1
2
"error"
"ReferenceError: i is not defined
```

  - `const` ตัวแปรระดับ Block ที่เป็นค่าคงที่

JavaScript ES6 : Data Type



- **Data Types (ชนิดของข้อมูล)** JavaScript เป็น Dynamic Types คือปรับเปลี่ยนไปตามค่าที่เก็บ
  - ตัวแปรเดิม สามารถใช้เก็บข้อมูลต่างชนิดกันได้

```
let x = "Hello World!!"
x = 5 + 3;
x = true
```

- JavaScript มี 8 Data types
  - String
  - Number
  - Bigint
  - Boolean
  - Undefined
  - Null
  - Symbol
  - Object

```
// Number
let amount = 20;
let price = 2.25;

// String
let name = "Helen";
let lastName = 'Smith';

// Booleans
let x = true;
let y = false;
```

String จะใช้
“ ” หรือ ‘ ’ คร่อมก็ได้

JavaScript ES6 : Data Type



- JavaScript Object Datatypes อาจบรรจุ
  - Object

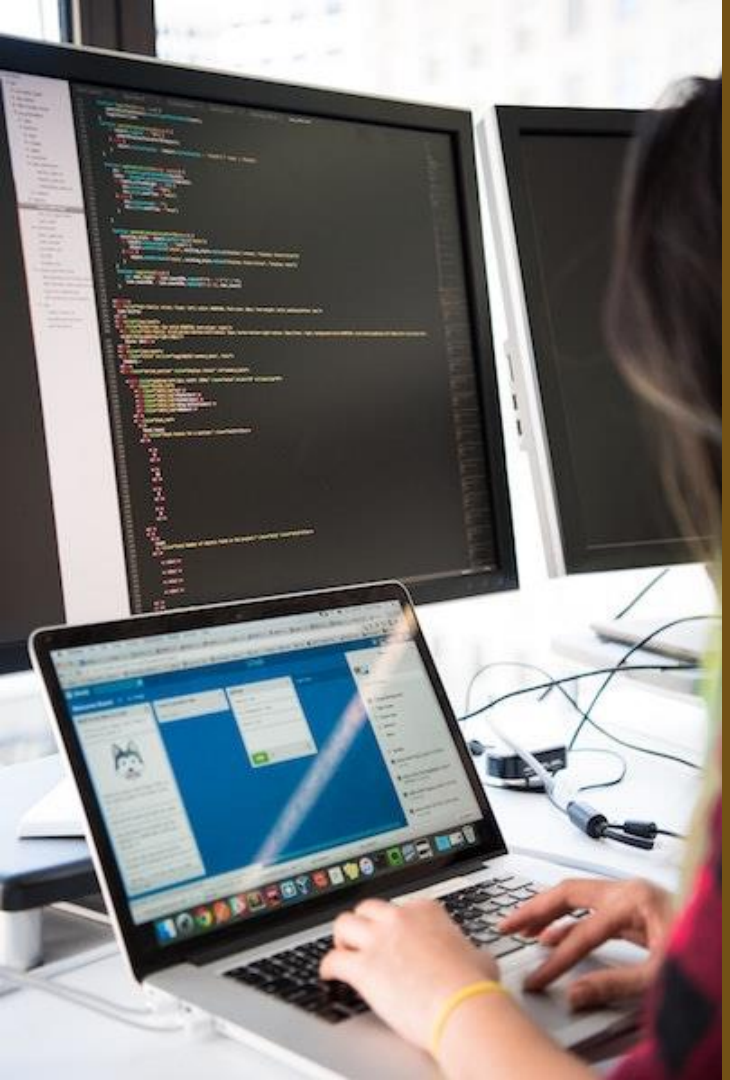
```
const p1 = {product:"Honey", price:34.55};
```

  - Array

```
const products = ["Milk", "Butter", "Egg"];
```

  - Date

```
const d = new Date(2023, 12, 20);
```



Workshop 2.1 :
Writing Basic JavaScript ES6

Environment Setup



NodeJS

- ติดตั้ง Node Version 18.7+ โดย Node มาพร้อมกับ Node Package Manager (npm)
  - Node.js LTS v20.16: https://nodejs.org/dist/v20.16.0/node-v20.16.0-x64.msi
  - npm version 5.2 ขึ้นไป มาพร้อมกับ npx (execute npm package binaries) ซึ่งทำให้สามารถติดตั้ง node package ต่างๆได้สะดวกมากยิ่งขึ้น

```
PS C:\Users\ongtang> node -v
v20.16.0
```

Environment Setup

- ติดตั้ง VS Code Extension ที่เป็นประโยชน์ต่อการทำงานกับ JavaScript

Extension เพื่อช่วยในการพัฒนา Code

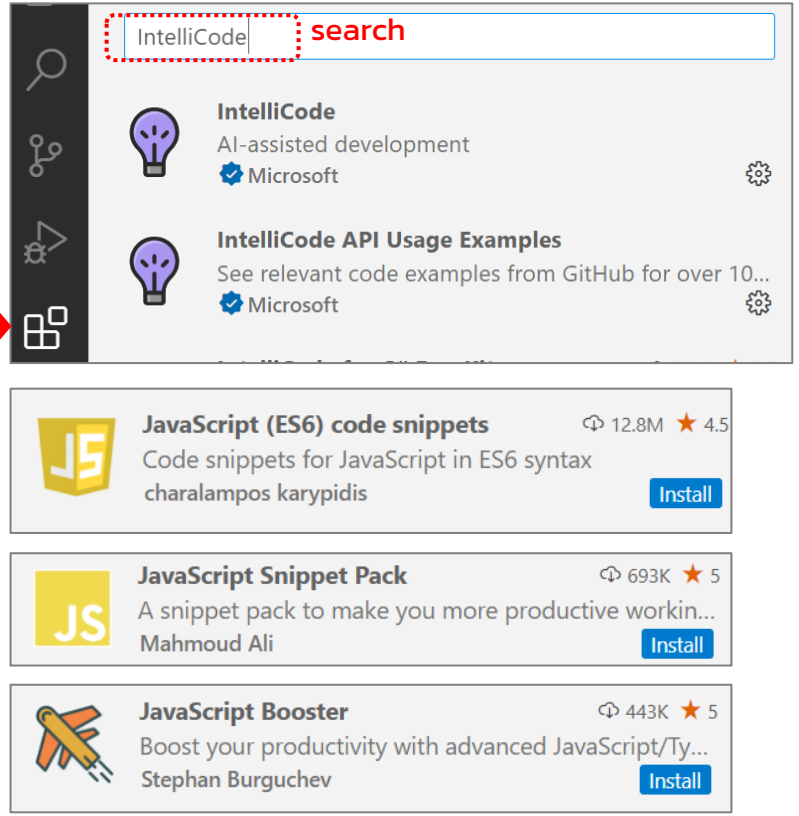
- IntelliCode
- JavaScript (ES6) code snippets
- JavaScript Snippet Pack
- JavaScript Booster

Extension เพื่อช่วยในการ Run Code

- Code Runner



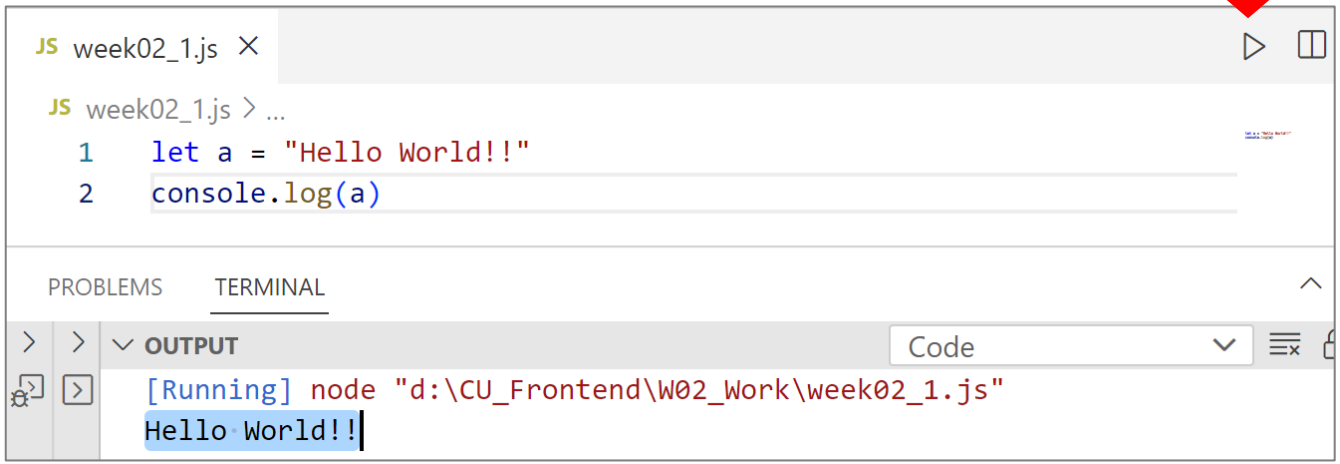
extension



Try Writing JavaScript

- สร้างโฟลเดอร์ `W02_JS` สำหรับจัดเก็บไฟล์ ใช้ VS Code เปิดโฟลเดอร์ดังกล่าว
  - สร้างไฟล์ `week02_1.js` บรรจุ code ดังภาพ
  - กดปุ่ม Run เพื่อแสดงผลผ่านเฟรม Terminal

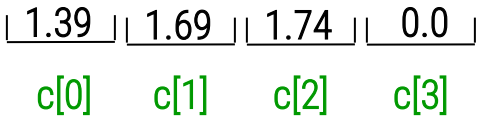
ปุ่ม Run จาก Code Runner Extension



JavaScript ES6 : Array



- Array คือ โครงสร้างข้อมูลที่นำตัวแปรมาวางเรียงต่อกันในหน่วยความจำ
- เรียกตัวแปรแต่ละตัวว่า สมาชิกของ array (elements)
- การเข้าถึงข้อมูลสามารถอ้างอิงโดยใช้ ดัชนี (index) ซึ่งเป็นตัวเลขจำนวนเต็ม



c = array name

หากไม่ใช้งาน array :

```
let product1 = "Milk";
let product2 = "Butter";
let product3 = "Egg";
```

ใช้งาน array :

```
const products = ["Milk", "Butter", "Egg"];
```

JavaScript ES6 : Array



- รูปแบบการประกาศตัวแปรแบบ array:
  - Empty Array

```
variable_keyword array_name = [];
```

```
variable_keyword array_name = new Array();
```

```
let students = [];
let courses = new Array();
```

  - ประกาศพร้อมระบุสมาชิกตั้งต้น

```
variable_keyword array_name = [m1,m2, …];
```

```
variable_keyword array_name = new Array(m1,m2, …);
```

```
let students = ["Peter", "Alice", "Mathew"];
let preferences = new Array("Sport", 45, 24.65, true);
```

JavaScript Array รองรับการจัดเก็บข้อมูลที่มี Data Type ต่างกัน

JavaScript ES6 : Array



- การเข้าถึงข้อมูลใน array:

```
array_name[index]
```

  - *index* = ตัวเลขจำนวนเต็ม
  - 0 <= *index* < *size*

```
let preferences = new Array("Sport", 45, 24.65, true);
preferences[3] = "Musics";
console.log(preferences[0]);
console.log(preferences[1] + preferences[2]);
console.log(preferences[3]);
```

```
Sport
69.65
Musics
```

- การเพิ่มข้อมูลลงใน array

```
array_name.push(member)
```

```
let students = [];
students.push("Jason");
console.log(students);
```

```
[ 'Jason' ]
```

- การหาขนาดของ array

```
array_name.length
```

```
let preferences = new Array("Sport", 45, 24.65, true);
console.log(preferences.length);
```

```
4
```

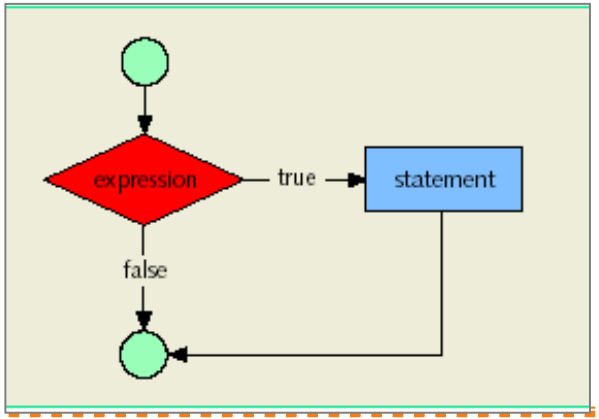
JavaScript ES6 : Control Structure

- การเลือกทางเดียว (One-Way Selection)

เงื่อนไขที่ใช้ในการตัดสินใจ

```
if(expression)
{
   statement1;
   statement2;
   …
}
```

ชุดคำสั่งที่จะให้ทำงาน
เมื่อเงื่อนไขถูกต้อง




```
if(expression)
   statement1;
```

ถ้าคำสั่งมีเพียงคำสั่งเดียว
ไม่จำเป็นต้องมี { }

```
let total = 30.25;
if(total>30)
{
    console.log("Receive Discount");
    total = total * 0.95;
}
console.log("Total = " + total);
```

```
if(username=="member01")
    console.log("Welcome");
```

JavaScript ES6 : Control Structure

- การเลือก 2 ทาง (Two-Way Selection)

เงื่อนไขที่ใช้ในการตัดสินใจ

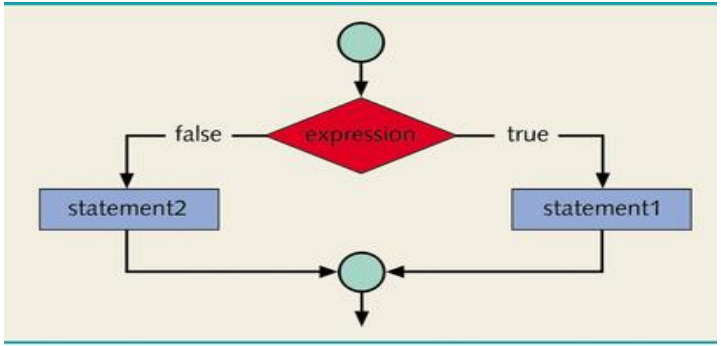
```
if(expression)
{
   statement 1;
   statement 2;
   …
}
else
{
   statement a;
   statement b;
   …
}
```

ชุดคำสั่งที่จะให้ทำงาน เมื่อเงื่อนไขถูกต้อง

ชุดคำสั่งที่จะให้ทำงาน เมื่อเงื่อนไขผิด

```
if(expression)
   statement 1;
else
   statement a;
```

ถ้าคำสั่งมีเพียงคำสั่งเดียว ไม่ต้องมี { } ก็ได้

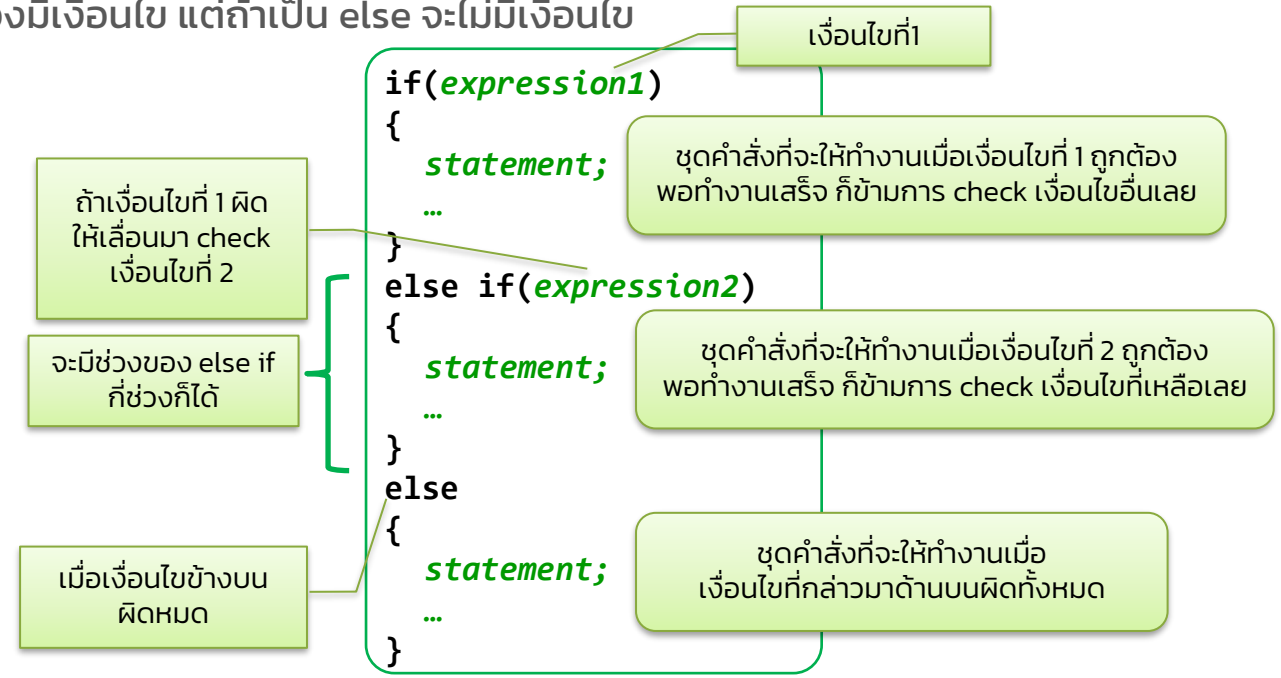



```
let total = 25.25;
if(total>30)
{
    console.log("Receive Discount");
    total = total * 0.95;
}
else
{
    console.log("Shop more to receive discount");
}
console.log("Total = " + total);
```

```
Shop more to receive discount
Total = 25.25
```

JavaScript ES6 : Control Structure

- เลือกได้หลายทาง
  - ในกรณีที่มีทางเลือกมากกว่าสองทาง เราต้องใช้ if เข้ามาซ้อนเอาไว้เพื่อให้ทำเงื่อนไขต่อไป
  - ข้อสังเกตุ else if จะต้องมีเงื่อนไข แต่ถ้าเป็น else จะไม่มีเงื่อนไข



JavaScript ES6 : Control Structure



การทำงานแบบวนซ้ำ (Iteration)

- สั่งให้ code บางชิ้นของโปรแกรมทำงานแบบวนซ้ำ ตามเงื่อนไขที่กำหนด
  - ลดความยาว code และเวลาในการเขียนโปรแกรม
  - สามารถพิจารณาได้ว่า code ชิ้นนั้นๆทำงานไปแล้วกี่รอบ

- รูปแบบคำสั่งในการเขียน code เพื่อทำงานวนซ้ำ
  - โครงสร้างแบบ `for`
  - โครงสร้างแบบ `for in`
  - โครงสร้างแบบ `for of`
  - โครงสร้างแบบ `while`
  - โครงสร้างแบบ `do … while`

JavaScript ES6 : Control Structure

- **โครงสร้างแบบ For**
  - ให้เริ่มทำงานและทำไปตราบใดที่สถานะเงื่อนไขที่กำหนดยังเป็นจริงอยู่

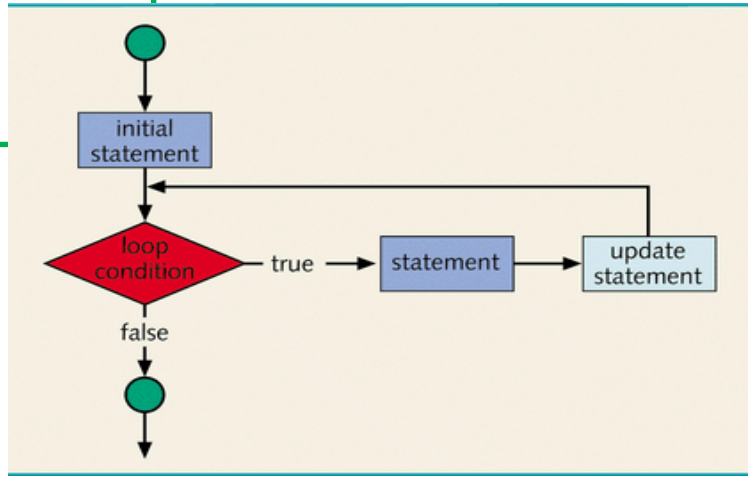
สถานะเริ่มต้น | สถานะเงื่อนไข | การ update สถานะ

```
for(initial statement; loop condition; update statement)
{
  statement1;
  statement2;
}
```

คำสั่งที่จะให้วนทำงาน

```
for(let i=0; i<5; i++)
{
    console.log("Number is " + i);
}
```

```
Number is 0
Number is 1
Number is 2
Number is 3
Number is 4
```



JavaScript ES6 : Control Structure



- **โครงสร้างแบบ For**

```
for(let i=0, n=3; i<5; i++, n++)
{
    console.log("i is " + i + ", n is " + n);
    console.log("Multiplied to " + i*n);
}
```

```
i is 0, n is 3
Multiplied to 0
i is 1, n is 4
Multiplied to 4
i is 2, n is 5
Multiplied to 10
i is 3, n is 6
Multiplied to 18
i is 4, n is 7
Multiplied to 28
```

- Variable Scope

var ประกาศไว้ใน loop แต่เป็น global / function scope จึงใช้งานนอก loop ได้

```
for(var i=0; i<5; i++)
{
    console.log("Number is " + i);
}
console.log("Final i is " + i);
```

```
Number is 0
Number is 1
Number is 2
Number is 3
Number is 4
Final i is 5
```

JavaScript ES6 : Control Structure



- ระวัง Variable Scope!!

let ประกาศ block scoped
จึงใช้งานได้แค่ใน loop

```
for(let i=0; i<5; i++)
{
    console.log("Number is " + i);
}
console.log("Final i is " + i);
```

```
Number is 0
Number is 1
Number is 2
Number is 3
Number is 4
d:\CU_Frontend\W02_Work\tempCodeRunnerFile.js:5
console.log("Final i is " + i);
                            ^

ReferenceError: i is not defined
```

เป็นการ redeclare

```
var i=25;
for(var i=0; i<5; i++)
{
    console.log("Number is " + i);
}
```

อยู่คนละ scope

```
let i=25;
for(let i=0; i<5; i++)
{
    console.log("Number is " + i);
}
```

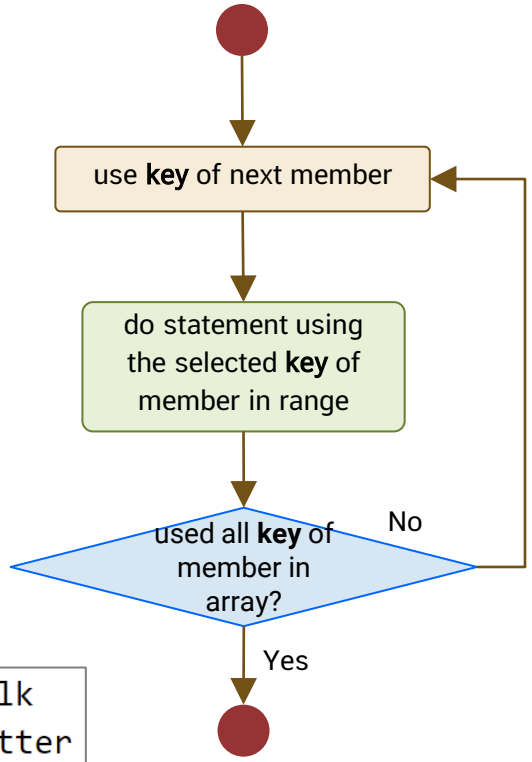
JavaScript ES6 : Control Structure

- **โครงสร้างแบบ For In**
  - วนเข้าถึง key ของ สมาชิกของ array ด้วย for in

ตัวแปรเก็บค่า index ของสมาชิกใน array โดยเปลี่ยนไปในแต่ละรอบ

```
for(index in array)
{
   statement1;
   statement2;
}
```

คำสั่งที่จะให้วนทำงาน




```
const products = ["Milk", "Butter", "Egg"];
for(let i in products)
{
    console.log("Product is " + products[i]);
}
```

```
Product is Milk
Product is Butter
Product is Egg
```

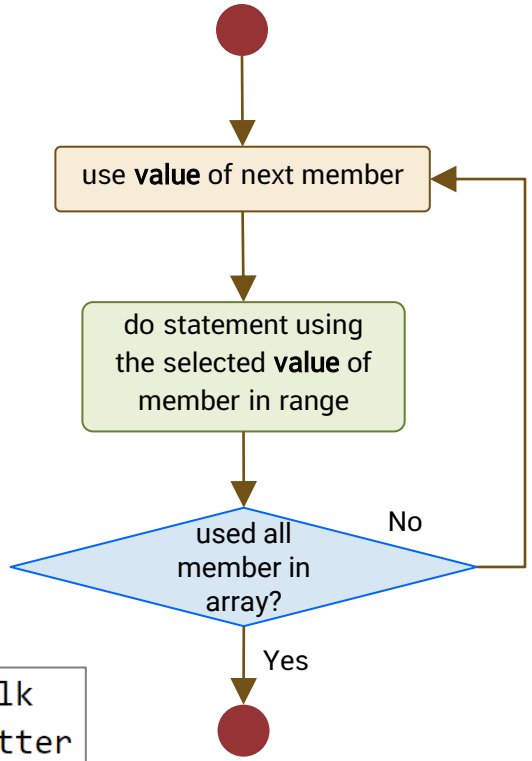
JavaScript ES6 : Control Structure

- **โครงสร้างแบบ For Of**
  - วนเข้าถึงสมาชิกของ array ด้วย for of

ตัวแปรเก็บค่า value ใน array โดยเปลี่ยนไปในแต่ละรอบ

```
for(member of array)
{
   statement1;
   statement2;
}
```

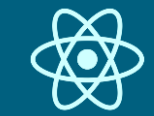
คำสั่งที่จะให้วนทำงาน



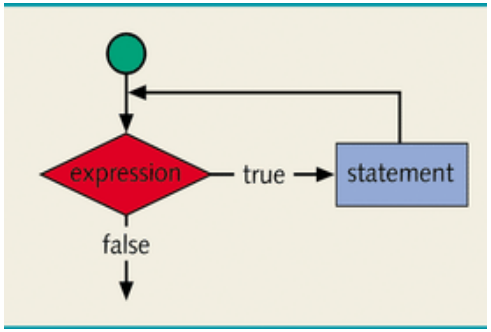

```
const products = ["Milk", "Butter", "Egg"];
for(let p of products)
{
    console.log("Product is " + p);
}
```

```
Product is Milk
Product is Butter
Product is Egg
```

JavaScript ES6 : Control Structure



- **โครงสร้างแบบ while**
  - คำสั่ง while จะวนทำงานตราบเท่าที่เงื่อนไขเป็นจริง
  - จะหยุดทำงานซ้ำ/หลุดลูป เมื่อเงื่อนไขกลายเป็นเท็จ
  - ถ้าเงื่อนไขไม่มีค่าเป็นเท็จ Code ส่วนนั้นจะทำงานวนไปเรื่อยๆ (infinite loop)



สถานะเงื่อนไข

```
while(condition)
{
   statement1;
   statement2;
}
```

คำสั่งที่จะให้วนทำงาน

- มักมีการอับเดตเงื่อนไขภายในลูป

ก่อนเริ่ม loop ให้ค่า n = 50

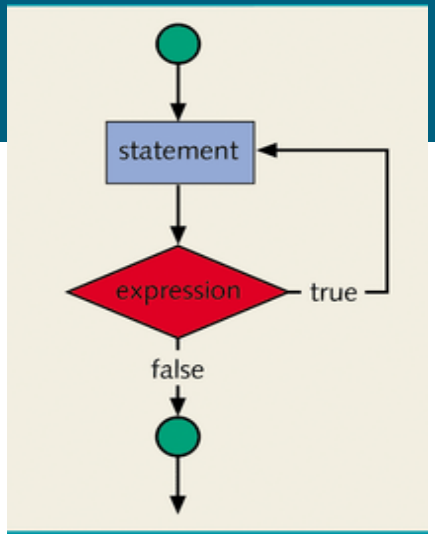
```
var n = 50;
while(n>10)
{
    n = n/2;
    console.log("n = " + n);
}
```

ทำงานตราบเท่าที่ n มากกว่า 10

การทำงานแต่ละรอบ ลดค่า n ลงครึ่งหนึ่ง

```
n = 25
n = 12.5
n = 6.25
```

JavaScript ES6 : Control Structure



- โครงสร้างแบบ do ... while
  - สำหรับคำสั่ง do...while จะเริ่มวนทำงานต่อเมื่อเงื่อนไข (condition) เป็นจริง
  - จะหยุดทำงานซ้ำ/หลุดลูป เมื่อเงื่อนไขเป็นเท็จ
  - ทำงานภายในลูปก่อนหนึ่งครั้ง ก่อนที่จะทำการตรวจสอบเงื่อนไข

เริ่มทำรอบแรกเลย
โดยไม่ check เงื่อนไข

```
do
{
   statement1;
   statement2;

} while(condition);
```

คำสั่งที่จะให้วนทำงาน

Check เงื่อนไข
หลังจากจบรอบแรก

สถานะเงื่อนไข

กำหนดค่าเริ่มต้นของ q กับ n

เริ่มทำรอบแรกเลย
โดยไม่ check
เงื่อนไข

```
var q = 5, n = 20;
do {
    q += 2;
    console.log("q=" + q + ", n=" + n);
} while(q<=n);
```

เงื่อนไขการทำงาน
สำหรับรอบถัดไป

```
q=7, n=20
q=9, n=20
q=11, n=20
q=13, n=20
q=15, n=20
q=17, n=20
q=19, n=20
q=21, n=20
```

JavaScript ES6 : Control Structure



- คำสั่ง break (break Statements)
  - เมื่อต้องการออกจากลูป
  - หลังจากคำสั่ง break โปรแกรมจะทำงานในคำสั่งถัดจากคำสั่งที่ออกมา

```
while (a < b)
{  statement1
   break
}
statement3;
```

```
const products = ["Milk", "Butter", "Egg"];
for(let p of products)
{
    if(p=="Butter")
        break;
    console.log("Product is " + p);
}
console.log("-- End --");
```

```
Product is Milk
-- End --
```

JavaScript ES6 : Control Structure

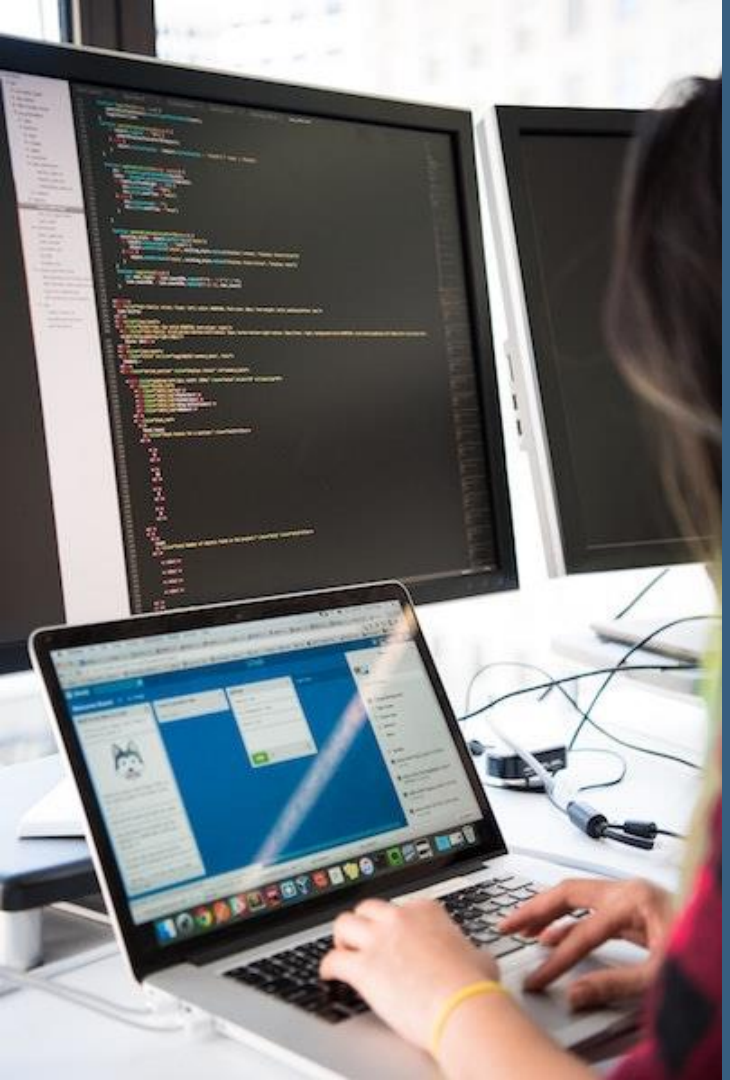


- คำสั่ง continue (continue Statements)
  - เมื่อต้องการทำงานภายในลูป ต่อเนื่องไป
  - หลังจากคำสั่ง continue โปรแกรมจะกลับไปทำงานในส่วนของการตรวจสอบเงื่อนไข

```
while (a < b)
{  statement1;
   continue;
}
statement3;
```

```
const products = ["Milk", "Butter", "Egg"];
for(let p of products)
{
    if(p=="Butter")
        continue;
    console.log("Product is " + p);
}
console.log("-- End --");
```

```
Product is Milk
Product is Egg
```

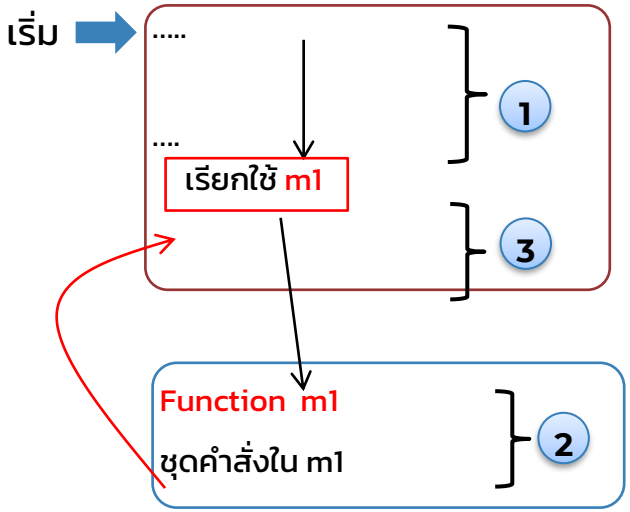


Function

ฟังก์ชั่น (Function)

- **ฟังก์ชั่น (Function)** คือชุดคำสั่งที่เราใช้บ่อยๆ
  - เมื่อรวบรวมชุดคำสั่งไว้เป็นฟังก์ชั่น ก็จะเรียกใช้ง่าย ไม่ต้องเขียน code เดิมซ้ำบ่อยๆ

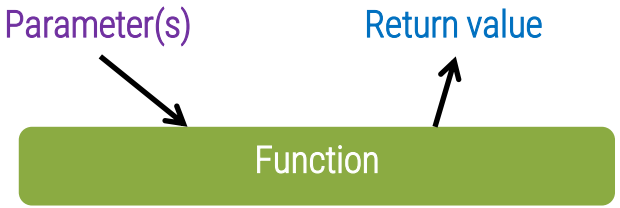
  - ชุดคำสั่งที่อยู่ในฟังก์ชั่น จะเริ่มทำงาน ณ. จุดที่มีการเรียกใช้ฟังก์ชั่น
  - เมื่อทำชุดคำสั่งในฟังก์ชั่นจบ โปรแกรมจะกลับมาทำงานต่อจาก จุดที่มีการเรียกใช้ฟังก์ชั่น



ฟังก์ชั่น (Function)

- วิธีการดำเนินการบนข้อมูล เพื่อให้ได้มาซึ่งคำตอบของปัญหา
  - เราสามารถส่งข้อมูลที่ใช้เป็น input ของฟังก์ชั่นเรียกว่า**พารามิเตอร์ (parameter)** ไปให้ฟังก์ชั่นนั้น
  - เราสามารถรอรับค่า output ซึ่งคือ**ผลลัพธ์ (return value)** ที่ได้จากการดำเนินการ

- Function มี parameters ได้หลายค่า
- ค่า return value มีได้ค่าเดียว หรือไม่คืนค่า



JavaScript ES6 : Function



- ฟังก์ชั่นที่ไม่มีการคืนค่า ไม่มีอาร์กิวเมนต์

```
function Function_name()
{
    ...
    ชุดคำสั่งในฟังก์ชั่น
    ...
}
```

- ***Function_name*** = ชื่อฟังก์ชั่น

```
var weight = 55;
var height = 168;
bmiCalculation();

function bmiCalculation()
{
    let bmi = weight / ((height/100)*(height/100));
    console.log("BMI = " + bmi);
}
```

```
BMI = 19.48696145124717
```

JavaScript ES6 : Function



- ฟังก์ชั่นที่ไม่มีการคืนค่า มีอาร์กิวเมนต์

```
function Function_name(arg1, arg2, …)
{
    …
    ชุดคำสั่งในฟังก์ชั่น
    …
}
```

- **Function_name** = ชื่อฟังก์ชั่น
- **arg1, arg2, …**
  = อาร์กิวเมนต์ที่ฟังก์ชั่นรับ parameter เข้ามา

```
bmiCalculation(55, 168);

function bmiCalculation(weight, height)
{
    let bmi = weight / ((height/100)*(height/100));
    console.log("BMI = " + bmi);
}
```

```
BMI = 19.48696145124717
```

JavaScript ES6 : Function



- ฟังก์ชั่นที่มีการคืนค่า ไม่มีอาร์กิวเมนต์

```
function Function_name()
{
   ชุดคำสั่งในฟังก์ชั่น
   ...
   return return_value;
}
```

- **Function_name** = ชื่อฟังก์ชั่น
- **return_type** = ชนิดข้อมูลของค่าที่คืนออกมา
- **return_value** = ค่าที่คืนออกมา

```
let weight = 55;
let height = 168;

let myBmi = bmiCalculation();
console.log("My BMI = " + myBmi);

function bmiCalculation()
{
    let bmi = weight / ((height/100)*(height/100));
    return bmi;
}
```

```
My BMI = 19.48696145124717
```

JavaScript ES6 : Function



- ฟังก์ชั่นที่มีการคืนค่า มีอาร์กิวเมนต์

```
fun Function_name(arg1, arg2, …)
{
   ชุดคำสั่งในฟังก์ชั่น
   …
   return return_value;
}
```

- `return_type` = ชนิดข้อมูลของค่าที่คืนออกมา
- `Function_name` = ชื่อฟังก์ชั่น
- `return_value` = ค่าที่คืนออกมา
- `arg1, arg2, …` = ชุดของอาร์กิวเมนต์

```
let myBmi = bmiCalculation(55, 168);
console.log("My BMI = " + myBmi);

function bmiCalculation(weight, height)
{
    let bmi = weight / ((height/100)*(height/100));
    return bmi;
}
```

```
My BMI = 19.48696145124717
```

JavaScript ES6 : Function



- **Arrow Functions**

  เป็นรูปแบบใหม่ในการประกาศฟังก์ชั่นของ Java Script

  - ตัวอย่าง เมื่อไม่มี argument

```
const greet = () => {
  return 'Hi!';
};
```

  - ตัวอย่าง เมื่อมี 1 argument

```
const greet = (name) => {
  return 'Hello, ' + name + '!';
};
```

  - ตัวอย่าง เมื่อมีหลาย arguments

```
const bmiCalculation = (weight, height)=> {
    let bmi = weight / ((height/100)*(height/100));
    return bmi;
}

console.log("My BMI = " + bmiCalculation(55, 168));
```

```
My BMI = 19.48696145124717
```

JavaScript ES6 : Function



- **การใช้งาน forEach บน Array เพื่อทำงานกับ Function**
  - วนเข้าถึงสมาชิกของ array ด้วย foreach

```
let prices = [45.43, 199, 1850.50];
prices.forEach(netPrice);

function netPrice(price)
{
    let afterVat = price * 1.07;
    console.log("Price with VAT = " + afterVat);
}
```

```
Price with VAT = 48.6101
Price with VAT = 212.93
Price with VAT = 1980.035
```

```
let prices = [45.43, 199, 1850.50];

prices.forEach((p) => {
    let afterVat = p * 1.07;
    console.log("Price with VAT = " + afterVat);
});
```

```
Price with VAT = 48.6101
Price with VAT = 212.93
Price with VAT = 1980.035
```

JavaScript ES6 : Array Method



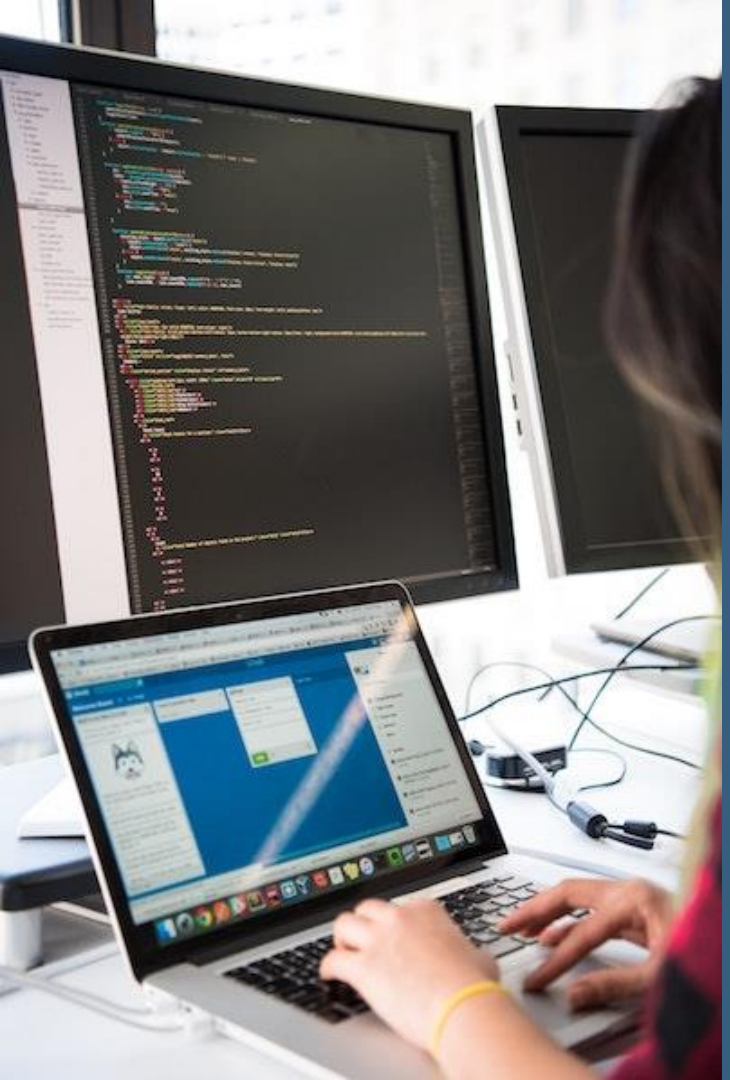
- Array Methods ตัวอย่างเช่น
  - `push()` เพิ่ม element ต่อท้าย array คืนค่าเป็น length ของ array ผลลัพธ์
  - `pop()` นำ element สุดท้ายออกจาก array คืนค่าเป็น element ดังกล่าว
  - `toString()` แปลง array เป็น String คั่นแต่ละ element ด้วย ,
  - `splice()` เพิ่ม element เข้าใน array หรือลบ element ออกจาก array

```
array_name.splice(start, deleteCount, item0, item1,…, itemN)
```

```
const products = ["Milk", "Butter", "Egg", "Sugar"];
products.pop();
products.push("Honey");
console.log(products.toString())

products.splice(2,1);   //delete 1 member at index 2
console.log(products.toString())
```

```
Milk,Butter,Egg,Honey
Milk,Butter,Honey
```



Object-Oriented

ออบเจกต์และคลาส (Object & Class)



คลาส (Class)

- เป็นนิยาม (Descriptor) หรือรายละเอียด (Specification) ของออบเจกต์
- เป็นเหมือน แม่แบบ (Blueprint) ของออบเจกต์ชนิดนั้นๆ

ออบเจกต์ (Object)

- เป็นองค์ประกอบหลักของการเขียนโปรแกรมเชิงวัตถุ
- เป็นตัวแทนของสิ่งต่างๆที่แสดงถึงขอบเขตของปัญหาที่กำลังพิจารณา
- ระบุข้อมูลที่เกี่ยวข้องและปฏิสัมพันธ์ระหว่างสิ่งเหล่านั้น
- ออบเจกต์ชนิดเดียวกันสามารถรวมอยู่ในคลาสเดียวกัน
- เรียกออบเจกต์ว่าเป็นอินสแตนซ์ (Instance) ของคลาส ออบเจกต์แต่ละอัน ประกอบด้วย
  - สิ่งบ่งชี้ (Identity) ที่แสดงความเป็นเอกลักษณ์ของแต่ละออบเจกต์
  - สถานะ (State) ที่แสดงถึงสภาพหรือเงื่อนไขปัจจุบันของออบเจกต์
  - พฤติกรรม (Behavior) ที่บ่งบอกถึงความสามารถของมัน

ออบเจกต์และคลาส (Object & Class)

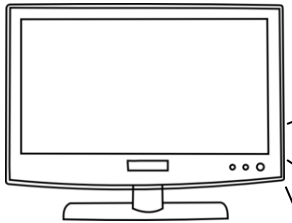


คลาสนิยามออบเจกต์ และระบุองค์ประกอบของออบเจกต์

- **Data** – องค์ประกอบด้านข้อมูลในการแก้ปัญหาอย่างใดอย่างหนึ่ง เช่น ในการคำนวณเกรดนักศึกษา ข้อมูลที่เกี่ยวข้องได้แก่
  - รหัสนักศึกษา
  - คะแนนรวม
  - เกรด
- **Functions** – องค์ประกอบด้านการดำเนินการบนข้อมูล เพื่อให้ได้มาซึ่งคำตอบของปัญหา เช่น ในการคานวณเกรดนักศึกษา ฟังก์ชั่นที่เกี่ยวข้องได้แก่
  - การรับคะแนนรวมเข้ามา
  - การคำนวณเกรด
  - การแสดงเกรด

OOP รวมทั้ง data และ Functions ไว้ด้วยกัน โดยเป็นองค์ประกอบของออบเจกต์ (Object)

คลาส Television



Attribute
brand
model
Price
currentChannel
volLevel
screenSize

Function
setChannel
getChannel
increaseVol
decreaseVol

ออบเจกต์ tel1



Attribute
brand = panasonics
model = dsc2130
price = 14000
currentChannel = ThaiPBS
volLevel = 20
screenSize = 32

Function
setChannel
getChannel
increaseVol
decreaseVol

ออบเจกต์ tel2



Attribute
brand = samsung
model = pp45002
price = 13500
currentChannel = mono29
volLevel = 22
screenSize = 40

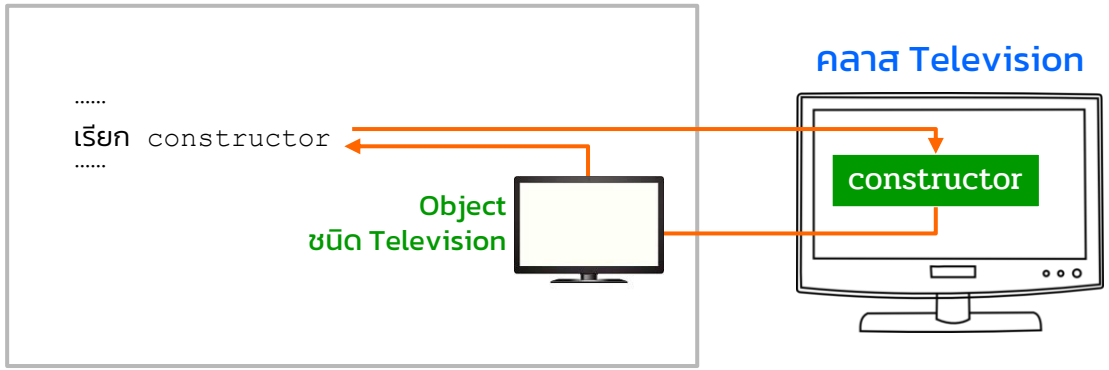
Function
setChannel
getChannel
increaseVol
decreaseVol

ออบเจกต์ tel3



Constructor

- แต่ละคลาสมี Function พิเศษที่เป็น constructors ใช้สำหรับสร้าง object ของคลาสนั้นๆ จาก blueprint ของ class
  - Constructor คืนค่ามาเป็น object ของคลาส นั้นๆ
- เราเรียกใช้ constructor เพื่อสร้าง object



JavaScript ES6 : Class & Object



- การสร้าง Object ผ่าน Object Literal
  - กำหนด Object Properties ในรูป property:value pair
  - เข้าถึง Object Property ด้วย *object*.property

```
let emp1 = {
    empId: "e001",
    name: "Robert",
    lastName: "Lee",
    position: "Engineer",
    salary: 45000
}

emp1.salary += 5000;
console.log(emp1.name + ' is ' + emp1.position);
console.log(emp1.name + ' salary is ' + emp1.salary);
```

```
Robert is Engineer
Robert salary is 50000
```

JavaScript ES6 : Class & Object



- การสร้าง Object ผ่าน Object Literal
  - กำหนด Object Methods ผ่าน Object Properties ที่เป็น function definition
  - เข้าถึง Object Method ด้วย *object*.functionCall

```
let emp1 = {
    empId: "e001",
    name: "Robert",
    lastName: "Lee",
    position: "Engineer",
    salary: 45000,
    bonus: function(months) {
        return this.salary*months;
    }
}

console.log(emp1.name + ' bonus is ' + emp1.bonus(3.5));
```

```
Robert bonus is 157500
```

JavaScript ES6 : Class & Object



- การสร้าง Object ผ่าน Object Constructor

```
function employee (empId, name, lastName, position, salary)
{
    this.empId = empId;
    this.name = name;
    this.lastName = lastName;
    this.position = position;
    this.salary = salary;
    this.bonus = function(months) {
        return this.salary*months;
    }
}

let e1 = new employee("e001", "Robert", "Lee", "Engineer", 45000);
console.log(e1.name + ' bonus is ' + e1.bonus(3.5));
```

```
Robert bonus is 157500
```

JavaScript ES6 : Class & Object



- ES6 สนับสนุน JavaScript Class โดยมีโครงสร้างเป็น

```
class ClassName {
  constructor() { ... }
}
```

```
class Employee {
    constructor(empId, name, lastName, position, salary)
    {
        this.empId = empId;
        this.name = name;
        this.lastName = lastName;
        this.position = position;
        this.salary = salary;
    }

    //Method
    bonus(months) {
        return this.salary*months;
    }
}

let e1 = new Employee("e001", "Robert", "Lee", "Engineer", 45000);
console.log(e1.name + ' bonus is ' + e1.bonus(3.5));
```

```
Robert bonus is 157500
```

JavaScript ES6 : Object

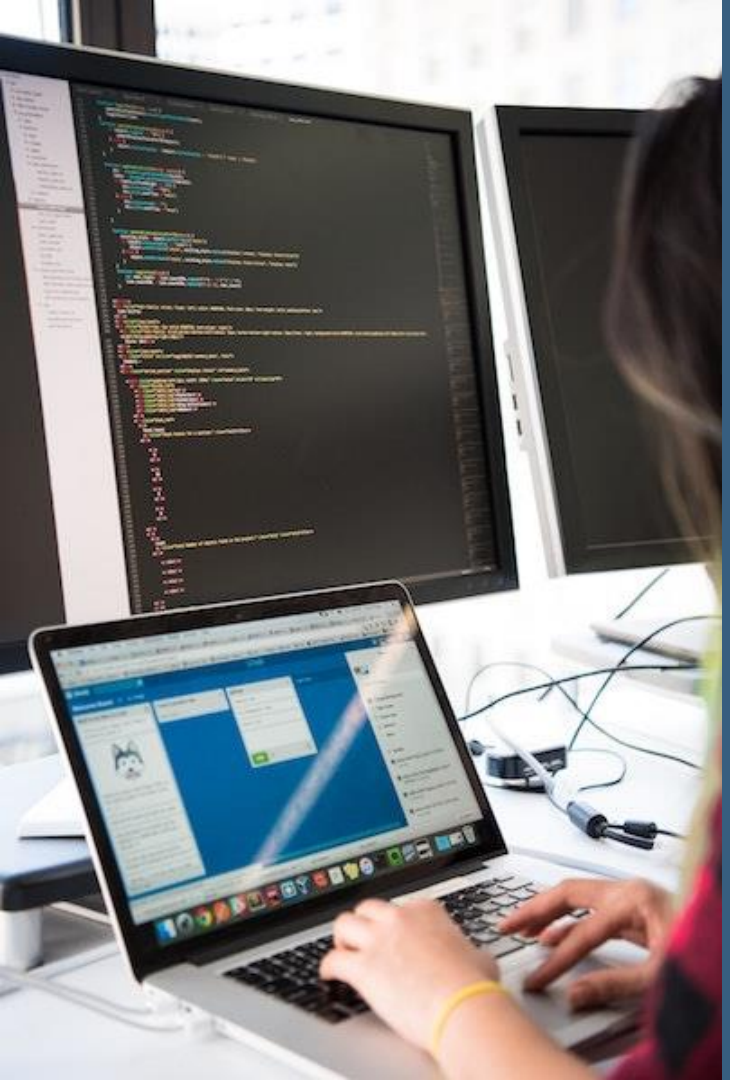


- การใช้งาน for in บน Object
  - วนเข้าถึงสมาชิกของ object ด้วย for in

```
class Employee {
    constructor(empId, name, lastName, position, salary)
    {
        this.empId = empId;
        this.name = name;
        this.lastName = lastName;
        this.position = position;
        this.salary = salary;
    }
    //Method
    bonus(months) {
        return this.salary*months;
    }
}

let e1 = new Employee("e001", "Robert", "Lee", "Engineer", 45000);
for(p in e1) {
    console.log(e1[p]);
}
```

```
e001
Robert
Lee
Engineer
45000
```



JavaScript in Web Browser

JavaScript ES6 กับ Web Browser



JavaScript สามารถเข้าถึง HTML Element บน HTML DOM และทำงานกับ Browser Object ได้

- การใส่ JavaScript เข้าไปใน HTML document
  - ใช้ tag <script> … </script> บรรจุ JavaScript โดยนิยมใส่ไว้ภายใน Tag <head> … </head> หรือไว้ในตอนล่างของ Tag <body> … </body>

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Hello Week 2</title>
<script>
    function greeting()
    {
        console.log("Week2 class");
    }
</script>
</head>
<body>
    <h1>Hello World!!</h1>
```

  - อาจระบุชนิดของ script เช่น <script type="text/javascript"> … </script>

JavaScript ES6 กับ Web Browser



JavaScript สามารถเข้าถึง HTML Element บน HTML DOM และทำงานกับ Browser Object ได้

- การใช้ External JavaScript
  - สร้างไฟล์ JavaScript `.js` แยกออกมา
  - ใช้ tag `<script src="..."></script>` ระบุไฟล์ `.js` นั้น ไว้เป็นค่าของ attribute `src`

ไฟล์ mainscript.js

```
function greeting()
{
    console.log("Week2 class");
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Hello Week 2</title>
<script src="mainscript.js"></script>
</head>
<body>
    <h1>Hello World!!</h1>
```

JavaScript ES6 กับ Web Browser



- ตัวอย่างการทำงานกับ Browser Object ด้วย JavaScript
  - การใช้งาน Alert Box

```
alert(message);
```

    - *message* = ข้อความที่ต้องการให้แสดงใน pop-up alert box
  - การใช้งาน Prompt Box เพื่อรับ input จากผู้ใช้

```
prompt(message, default-text)
```

    - *message* = ข้อความที่ต้องการให้แสดงใน prompt box
    - *default-text* = ข้อความ default
    - Return ค่าที่ user input เข้าไป
  - การใช้งาน Confirm Box

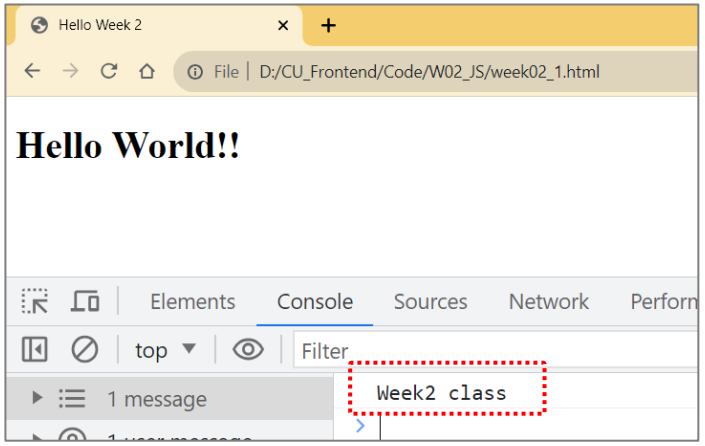
```
confirm(message)
```

    - *message* = ข้อความที่ต้องการให้แสดงใน confirm box
    - Return ค่าเป็น true หาก user กดปุ่ม OK ใน confirm box หรือเป็น false หาก user กดปุ่ม Cancel ใน confirm box

JavaScript ES6 กับ Web Browser

- การจัดการ HTML Events ด้วย JavaScript
  - นิยมใช้ HTML Event Attributes เช่น `onclick`, `onchange`, `onload`, `onunload`, `onmouseover`, `onmouseout` เป็นต้น
    - ตัวอย่างการแสดงข้อความออกทาง console log เมื่อโหลดหน้า

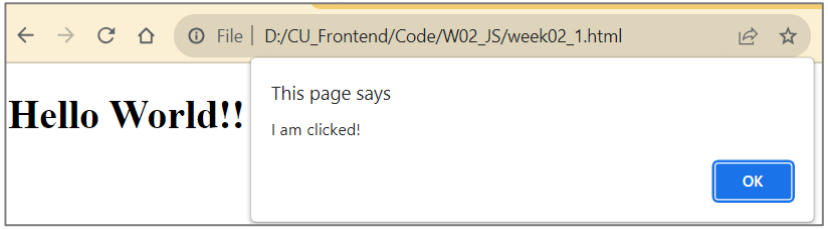
```
<title>Hello Week 2</title>
<script>
    function greeting()
    {
        console.log("Week2 class");
    }
</script>
</head>
<body onload="greeting();">
    <h1>Hello World!!</h1>
```



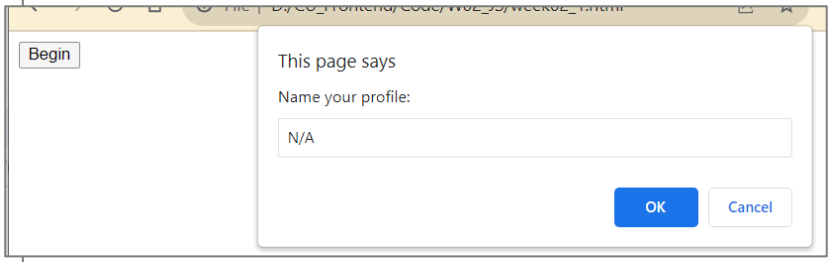
JavaScript ES6 กับ Web Browser



```
<h1 onclick="alert('I am clicked!')">Hello World!!</h1>
```




```
    }
    function apply()
    {
        let profileName = prompt("Name your profile:", "N/A");
        console.log(profileName);
    }
</script>
</head>
<body>
    <form>
        <input type="button" value="Begin" onclick="apply();">
    </form>
```



JavaScript ES6 กับ Web Browser



- **การเข้าถึง HTML Elements ด้วย JavaScript**
  - เข้าถึง HTML Element จาก id

```
document.getElementById(id)
```

    - *id* = ค่า id ของ element ที่ต้องการเข้าถึง
  - เข้าถึง HTML Element จาก class

```
document.getElementsByClassName(className)
```

    - *className* = ค่า class ของ element ที่ต้องการเข้าถึง

  - เข้าถึง HTML Element จากชื่อ tag

```
document.getElementsByTagName(tagName)
```

    - *tagName* = ค่า tag ของ element ที่ต้องการเข้าถึง



- การปรับเปลี่ยนค่าของ HTML Elements ด้วย JavaScript
  - เข้าถึงค่า content ใน HTML element

    ```
    element.innerHTML
    ```

    - *element* = HTML element ที่ทำงานด้วย

  - เข้าถึงค่า attribute ของ HTML element

    ```
    element.attributeName
    ```

    - *element* = HTML element ที่ทำงานด้วย
    - *attributeName* = ชื่อของ attribute ที่ต้องการเข้าถึง

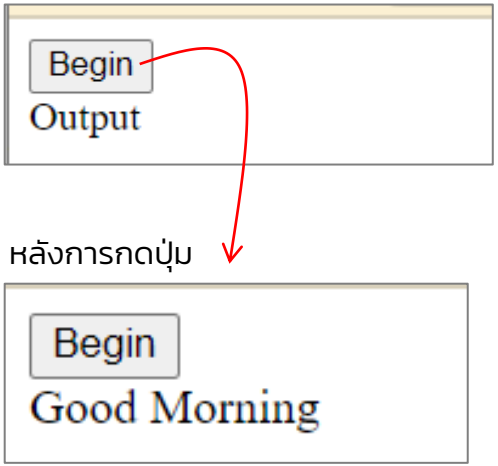
  - เข้าถึง style property ของ HTML element

    ```
    element.style.property
    ```

    - *element* = HTML element ที่ทำงานด้วย
    - *property* = ชื่อของ style property ที่ต้องการเข้าถึง

JavaScript ES6 กับ Web Browser

```
<script>
    function greeting()
    {
        let output = document.getElementById("output");
        output.innerHTML = "Good Morning"
    }
</script>
</head>
<body>
    <form>
        <input type="button" value="Begin" onclick="greeting();">
        <div id="output">Output</div>
    </form>
</body>
```



JavaScript ES6 กับ Web Browser



- การเข้าถึง HTML Form Elements ด้วย JavaScript
  - เข้าถึง Form Element จาก id

    ```
    document.getElementById(id)
    ```

    - *id* = ค่า id ของ element ที่ต้องการเข้าถึง

  - เข้าถึง Form จากชื่อ form และชื่อ form element

    ```
    document.forms["formName"]["formElementName"]
    ```

    - *formName* = ชื่อ form
    - *formElementName* = ชื่อ element ใน form

  - เข้าถึงค่าของ form element

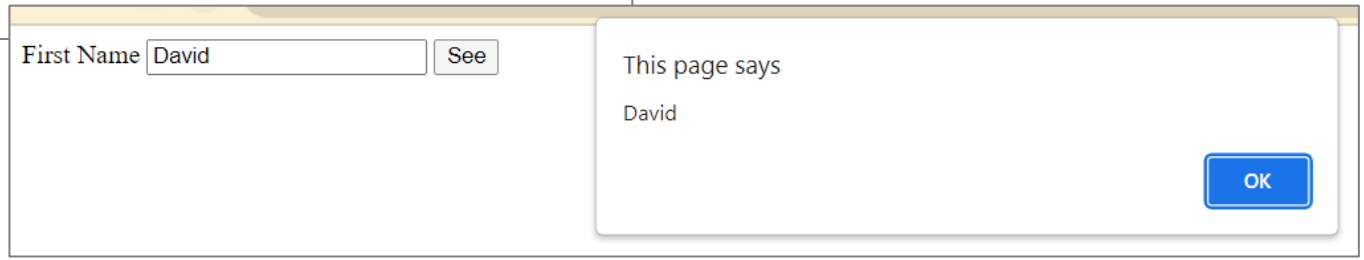
    ```
    formElement.value
    ```

    - *formElementName* = ชื่อ element ใน form

JavaScript ES6 กับ Web Browser



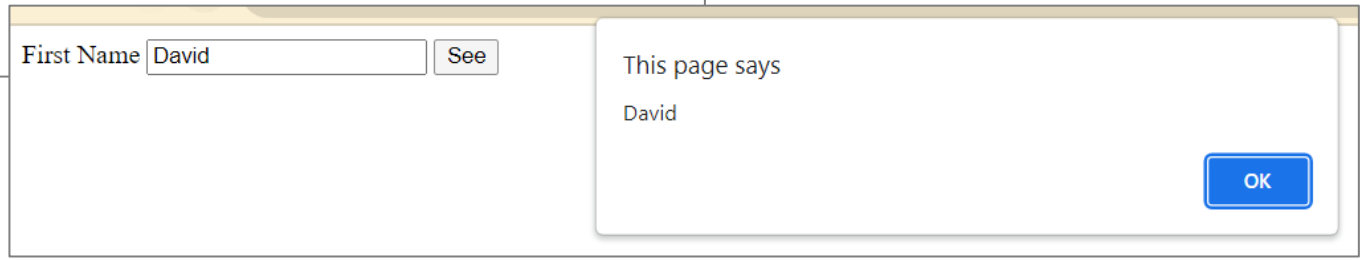
```
<script>
    function getData()
    {
        let name = document.getElementById("iFirstName");
        alert(name.value);
    }
</script>
</head>
<body>
    <form>
        <label for="iFirstName">First Name</label>
        <input type="text" name="firstName" id="iFirstName">
        <input type="button" value="See" onclick="getData();">
    </form>
</body>
```

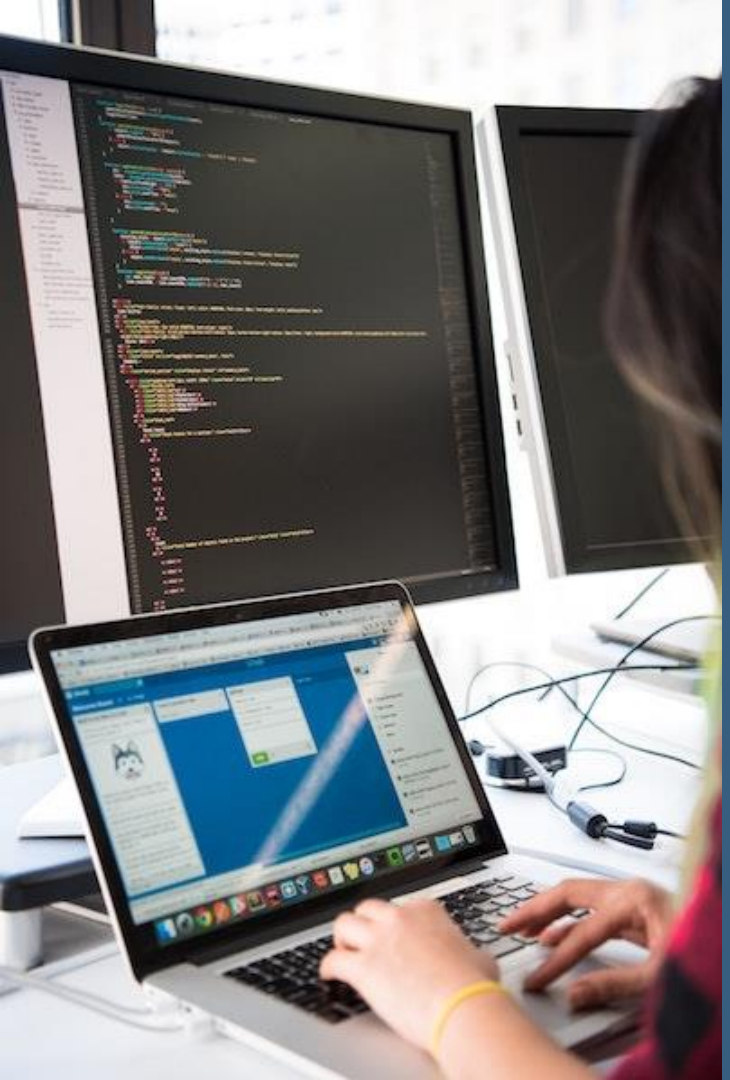


JavaScript ES6 กับ Web Browser



```
<script>
    function getData()
    {
        let name = document.forms["member"]["firstName"];
        alert(name.value);
    }
</script>
</head>
<body>
    <form name="member">
        <label for="iFirstName">First Name</label>
        <input type="text" name="firstName" id="iFirstName">
        <input type="button" value="See" onclick="getData();">
    </form>
</body>
```



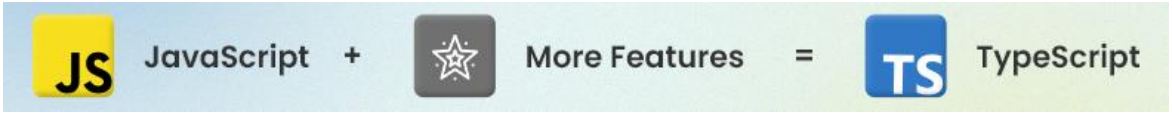


TypeScript (TS)

TypeScript (TS)



- TypeScript (.ts) เป็น superset ของ JavaScript โดยถูก compile ด้วย TypeScript Compiler (tsc) เป็น JavaScript
  - ทุก JavaScript สามารถเป็น TypeScript ได้ JavaScript ใช้งานกับ TypeScript ได้
    - เปลี่ยนนามสกุล จาก `.js` เป็น `.ts` จะกลายเป็น TypeScript และสามารถทำงานได้ตามปกติ
    - แต่ บาง JavaScript อาจไม่เป็น Valid TypeScript เนื่องจาก TypeScript มี Syntax ที่เข้มงวดกว่า
  - บาง TypeScript อาจไม่สามารถเป็น JavaScript ได้ เนื่องจาก JavaScript ไม่สนับสนุนบาง Features

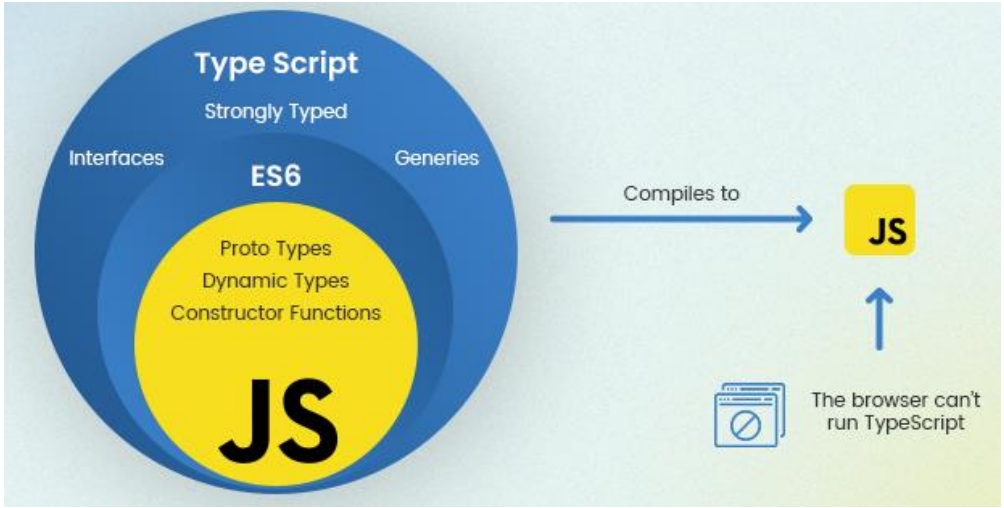


Source: *https://radixweb.com/blog/typescript-vs-javascript*

- TypeScript ถูกพัฒนาขึ้น เพื่อปิดข้อจำกัดบางอย่างของ JavaScript
  - JavaScript ถูกออกแบบมาเป็น client-side programming language ทั้ง ๆ ที่สามารถใช้เป็น server-side programming language แต่ด้วยขีดจำกัด ในการเป็น object-oriented programming ทำให้ไม่สามารถนำไปประยุกต์ใช้บน server ได้เต็มที่
  - TypeScript เติมเต็มข้อจำกัดของ JavaScript ทำให้สามารถเป็นได้ทั้ง client-side programming language และ server-side programming language

TypeScript (TS)

- **TypeScript** ได้รับการพัฒนาและขับเคลื่อนโดย Microsoft
  - เป็นภาษา open-source ที่ถูกออกแบบเป็น object-oriented programming
  - เป็นภาษาที่รับการ compile แบบ static ทำให้ตรวจจับ error ตั้งแต่ compile time สนับสนุน code ที่มีคุณภาพมากขึ้น
  - รองรับทั้ง dynamic และ static typing
  - รองรับการทำงานกับ classes, inheritance, interfaces, visibility scopes, unions, namespaces, modules ฯลฯ
  - เหมาะกับการใช้พัฒนา ทั้ง frontend และ backend



Source: *https://radixweb.com/blog/typescript-vs-javascript*

TypeScript (TS)



JavaScript and TypeScript compared		
Criteria	JavaScript	TypeScript
Trademark owner	Oracle	Microsoft
Standard	ECMAScript 5	ECMAScript 2015
Typing	Dynamic	Optional strong typing
Community and adoption	Larger than TypeScript	Smaller than JavaScript
Compilation	Not required	Compiled into JavaScript
File extension	.js and .jsx	.ts and .tsx
IDE support	Limited validation and refactoring capabilities	Extensive validation and refactoring capabilities
Complexity	Easy scripting language for new developers to learn	Additional features require an investment of time to learn
Readability	Large code bases are harder to read and decipher	Type system and object-oriented approach improve readability
Compatibility	All JavaScript code is also valid TypeScript code	TypeScript code gets compiled into fully compliant JavaScript

Source: *https://www.theserverside.com/tip/JavaScript-vs-TypeScript-Whats-the-difference*

TypeScript (TS): Static Typing



- รูปแบบการประกาศตัวแปร

```
variable_keyword   variable_name:data_type
```

let / var / const　　ชื่อตัวแปร　　ชนิดข้อมูล

- ตัวอย่าง
  - `let price:number;`
  - `let isValid:boolean;`
  - `let amount:number = 30;`
  - `let dept_name:string = “Accounting”;`

```
let department:string = "Marketing";
let member:number = 8;
console.log(department + " has " + member + " members");
```

```
Marketing has 8 members
```

TypeScript (TS): Static Typing



- **Implicit Type**
  - TypeScript สามารถอ้างอิงชนิดของข้อมูลจากค่าที่กำหนดให้กับตัวแปรได้
  - เราไม่จำเป็นต้องระบุ Data Type ให้กับตัวแปร
  - จะต้องมีการประกาศตัวแปรจึงจะสามารถเรียกใช้งานได้

```
let department = "Marketing";
let member = 8;
console.log(department + " has " + member + " members");
```

department ถูกมองว่าเป็นตัวแปร string โดยอัตโนมัติ
member ถูกมองว่าเป็นตัวแปร number โดยอัตโนมัติ

```
let member = 8;
member = "David";
```

member ถูกมองว่าเป็นตัวแปร number โดยอัตโนมัติ
เอาไปเก็บข้อความไม่ได้

TypeScript (TS): Static Typing



- **Type any**
  - ชนิดของข้อมูลเป็นอะไรก็ได้ เป็นการปิดกลไกตรวจสอบชนิดข้อมูล

```
let member:any;
member = "David";
member = 8
```

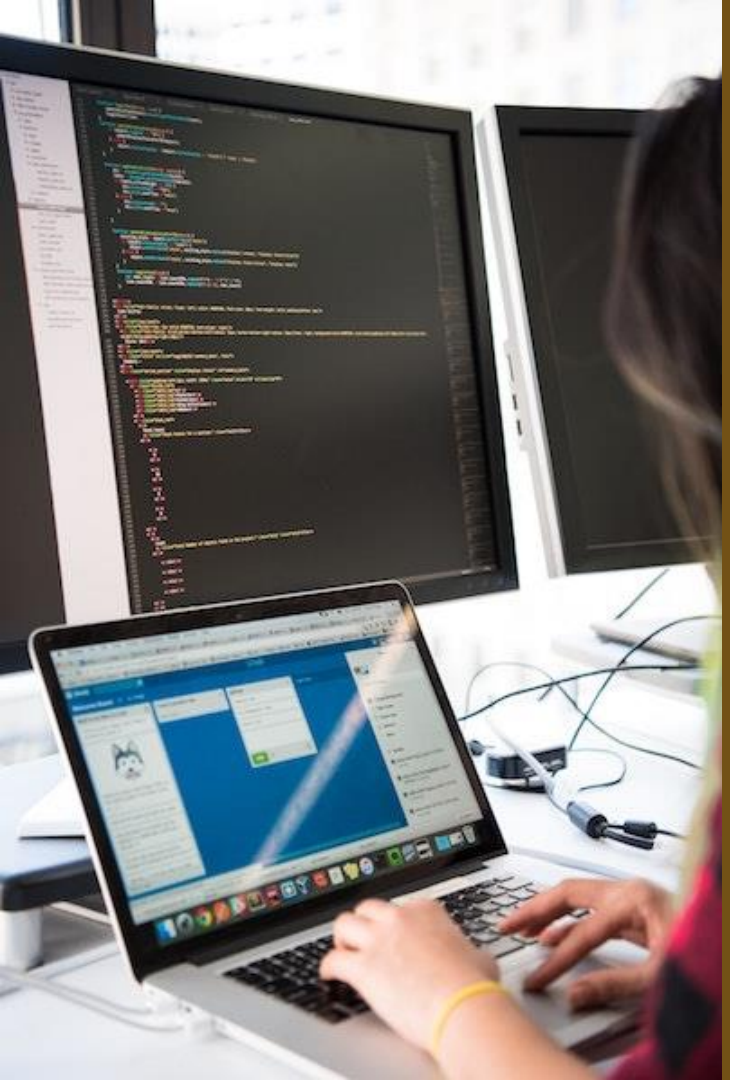
- **Type Union |**
  - ชนิดของข้อมูลสามารถเป็นได้มากกว่า 1 ชนิด

```
let member:string|number;
member = "David";
member = 8
```

- **Array**
  - รองรับ typing array ซึ่งเจาะจงชนิดข้อมูล

```
let member: string[] = []; //empty array
member.push("David");
member.push(1250.25);
```

```
let member: (string|number)[] = []; //empty array
member.push("David");
member.push(1250.25);
```



Workshop 2.2 :
Writing Basic TypeScript

Environment Setup



ติดตั้ง ts-node

- ts-node เป็น TypeScript execution engine สำหรับ Node ทำให้สามารถ run Typescript ได้โดยตรง โดยไม่ต้อง compile ไฟล์ Typescript(.ts) ให้เป็น Javascript(.js) ผ่าน `tsc` ก่อน

- ติดตั้ง ts-node แบบ global ผ่าน node package manager (npm)
- เปิด Power Shell หรือ Terminal
  - `npm install -g ts-node`

```
PS C:\Users\ongtang> npm install -g ts-node
```

Try Writing TypeScript

- สร้างโฟลเดอร์ `W02_TS` สำหรับจัดเก็บไฟล์ ใช้ VS Code เปิดโฟลเดอร์ดังกล่าว
  - สร้างไฟล์ `week02_ts1.ts` บรรจุ code ดังภาพ
  - กดปุ่ม Run เพื่อแสดงผลผ่านเฟรม Terminal



TypeScript (TS): Function



มีการใช้ Type กับทั้ง argument และ return value

- ฟังก์ชั่นที่ไม่มีการคืนค่า ไม่มีการรับอาร์กิวเมนต์จากภายนอก

```
function Function_name()
{
    ...
    ชุดคำสั่งในฟังก์ชั่น
    ...
}
```

```
let weight = 55;
let height = 168;
bmiCalculation();

function bmiCalculation() {
    let bmi = weight / ((height/100)*(height/100));
    console.log("My BMI = " + bmi);
}
```

TypeScript (TS): Function



มีการใช้ Type กับทั้ง argument และ return value

- ฟังก์ชั่นที่ไม่มีการคืนค่า มีการรับอาร์กิวเมนต์จากภายนอก

```
fun Function_name(arg1:arg1Type, arg2:arg2Type, …)
{
   …
   ชุดคำสั่งในฟังก์ชั่น
   …
}
```

- **Function_name** = ชื่อฟังก์ชั่น
- **arg1, arg2, …**
  = อาร์กิวเมนต์ของฟังก์ชั่น
- **arg1Type, arg2Type, …**
  = ชนิดข้อมูลของอาร์กิวเมนต์ฟังก์ชั่น

```
bmiCalculation(55, 168);

function bmiCalculation(weight:number, height:number) {
    let bmi = weight / ((height/100)*(height/100));
    console.log("My BMI = " + bmi);
}
```

TypeScript (TS): Function



- ฟังก์ชั่นที่มีการคืนค่า ไม่มีอาร์กิวเมนต์

```
fun Function_name() : return_type
{
   ชุดคำสั่งในฟังก์ชั่น
   …
   return return_value;
}
```

- `Function_name` = ชื่อฟังก์ชั่น
- `return_type` = ชนิดข้อมูลของค่าที่คืนออกมา
- `return_value` = ค่าที่คืนออกมา

```
let weight = 55;
let height = 168;
console.log("My BMI = " + bmiCalculation());

function bmiCalculation():number
{
    let bmi = weight / ((height/100)*(height/100));
    return bmi;
}
```

TypeScript (TS): Function



- ฟังก์ชั่นที่มีการคืนค่า ที่มีอาร์กิวเมนต์

```
fun Function_name(arg1:arg1Type, arg2:arg2Type, …) : return_type
{
   ชุดคำสั่งในฟังก์ชั่น
   …
   return return_value;
}
```

```
let weight = 55;
let height = 168;
console.log("My BMI = " + bmiCalculation(55, 168));

function bmiCalculation(weight:number, height:number):number
{
    let bmi = weight / ((height/100)*(height/100));
    return bmi;
}
```

TypeScript (TS): Class



- Member ของ class อันได้แก่ Attributes และ Methods ใน TypeScript ต้องการ Static Typing
- รองรับ visibility modifiers เพื่อระบุขอบเขตการมองเห็นของ Attributes และ Methods
  - public – เป็นสาธารณะ เข้าถึงได้จากภายนอก
  - private – เป็นส่วนตัวของ class
  - protected – เป็นส่วนตัวของ class และ class ที่สืบทอด (inherit) class นั้น ๆ

TypeScript (TS): Class



```
class Employee {

    public empId:string;
    private name:string;
    private lastName:string;
    private position:string;
    private salary:number;

    constructor(empId:string, name:string, lastName:string, position:string, salary:number)
    {
        this.empId = empId;
        this.name = name;
        this.lastName = lastName;
        this.position = position;
        this.salary = salary;
    }

    public bonus(months:number): number {
        return this.salary*months;
    }
}
```

```
let emp1 = new Employee("e001", "Robert", "Lee", "Engineer", 45000);
console.log(emp1.empId + " bonus is " + emp1.bonus(3.5));
```

TypeScript (TS): Class



```
class Employee {

    constructor(public empId:string, private name:string, private lastName:string,
        private position:string, private salary:number)
    {
        this.empId = empId;
        this.name = name;
        this.lastName = lastName;
        this.position = position;
        this.salary = salary;
    }

    public bonus(months:number): number {
        return this.salary*months;
    }
}
```

```
let emp1 = new Employee("e001", "Robert", "Lee", "Engineer", 45000);
console.log(emp1.empId + " bonus is " + emp1.bonus(3.5));
```